

Preguntas para el grupo F

1.

¿Cómo se definen estructuralmente los arreglos paralelos según el informe técnico?

☐ A.

Un contenedor orientado a objetos que encapsula atributos de texto y punto flotante en una sola entidad física.

☐ B.

Un tipo de dato compuesto embebido de forma nativa en los compiladores modernos de C++ y Java.

☐ C.

Una técnica de programación que declara dos o más arreglos independientes del mismo tamaño acoplados lógicamente por un índice común.

☐ D.

Una estructura de datos dinámica bidimensional que reasigna de manera automática celdas heterogéneas en el Heap.

Respuesta: C

2.

¿Cuál es el principal riesgo o vulnerabilidad de utilizar la técnica de arreglos paralelos al manipular los datos?

☐ A.

Un fallo inmediato por desbordamiento de pila (Stack Overflow) al cambiar el orden de los elementos.

☐ B.

La conversión forzada y automática de datos de punto flotante a cadenas de texto.

☐ C.

La activación prematura del Recolector de Basura (Garbage Collector) en entornos de ejecución estáticos.

☐ D.

La pérdida de integridad referencial y desincronización de índices al realizar ordenamientos o filtrados.

Respuesta: D

3.

De acuerdo al análisis comparativo de memoria del informe, ¿dónde almacena C++ un arreglo nativo declarado con una dimensión constante?

☐ A.

Dentro de un búfer virtual persistente administrado por el Recolector de Basura.

☐ B.

En una zona compartida híbrida entre el Stack y el descriptor nulo de registros.

☐ C.

Directamente en la Pila (Stack) de forma contigua, gestionada bajo una estructura LIFO.

☐ D.

En el Montículo (Heap) de manera dinámica a través del subsistema de la JVM.

Respuesta: C

4.

¿Qué ocurre en Java cuando se utiliza el operador new para instanciar un arreglo?

☐ A.

Se asigna un espacio estático en la CPU que contiene datos residuales previos denominados basura.

☐ B.

Se genera un puntero que inmediatamente sufre una degradación (pointer decay) perdiendo su dimensionalidad.

☐ C.

Se reserva un bloque de direcciones en el Montículo (Heap) y la JVM asegura la inicialización limpia de la memoria con valores por defecto.

☐ D.

El programa interrumpe el flujo arrojando una excepción `ArrayIndexOutOfBoundsException` de forma preventiva.

Respuesta: C

5.

En el contexto de las buenas prácticas expuestas, ¿por qué se desaconseja el uso de 'números mágicos' para definir el tamaño de los arreglos?

☐ A.

Porque reducen drásticamente la velocidad de cálculo del procesador al generar desplazamientos infinitos.

☐ B.

Porque representan una vulnerabilidad de mantenibilidad al estar hardcodeados, dificultando modificaciones futuras del código.

☐ C.

Porque obligan a que el ciclo do-while se ejecute de manera asíncrona dentro del Heap.

☐ D.

Porque provocan de forma obligatoria un error de sintaxis detectado en tiempo de compilación tanto en C++ como en Java.

Respuesta: B

6.

¿Cuál es la diferencia operativa fundamental entre C++ y Java cuando un bucle intenta acceder a una posición fuera de los límites de un arreglo?

☐ A.

Ambos lenguajes se comportan idénticamente delegando el control de los límites al Garbage Collector del sistema operativo.

☐ B.

C++ lanza inmediatamente un fallo Segmentation Fault controlado, mientras que Java corrompe variables vecinas en el Stack de forma silenciosa.

☐ C.

C++ puede permitir el acceso generando comportamiento indefinido o Buffer Overflow, mientras que Java detiene el flujo de forma segura lanzando una excepción.

☐ D.

C++ expande dinámicamente la memoria del Stack, en tanto que Java reduce el tamaño de la propiedad pública .length.

Respuesta: C

7.

En el sistema de filtrado de entrada implementado por el grupo, ¿qué rango de calificaciones se define como válido mediante la estructura repetitiva do-while?

☐ A.

De 0.0 a 10.0

☐ B.

De 1.0 a 7.0

☐ C.

De -1.0 a 1.0 según la escala del acumulador síncrono.

☐ D.

Cualquier valor numérico entero positivo menor a la constante MAX_ESTUDIANTES.

Respuesta: A

8.

De acuerdo a las reglas de negocio descritas en el informe, ¿cuál es la expresión relacional exacta para determinar que un estudiante ha aprobado?

☐ A.

"notas[i]>7.0"

☐ B.

"nombres[i]=="APROBADO""

☐ C.

"acumuladorNotas/dimension<7.0"

☐ D.

"notas[i]≥7.0"

Respuesta: D

9.

¿Cuál es la complejidad temporal del algoritmo de búsqueda lineal implementado para localizar a un estudiante y por qué?

☐ A.

O(1) debido a que la instrucción break reubica de forma inmediata los punteros de la pila en tiempo constante.

☐ B.

O(n) porque en el peor de los escenarios el tiempo de búsqueda depende directamente de la cantidad total de elementos a revisar uno por uno.

☐ C.

O(logn) ya que los arreglos se auto-ordenan mediante el método de la burbuja durante la captura síncrona.

☐ D.

O(n²) porque se requiere un doble bucle for anidado para cruzar de forma física el arreglo de nombres con el de notas.

Respuesta: B

10.

Al implementar la salida en pantalla del reporte, ¿qué herramientas específicas detalla el informe para lograr una visualización limpia alineada en columnas?

☐ A.

El calificador inmutable final de Java en conjunto con el ámbito determinista de la CPU de C++.

☐ B.

La secuencia de escape '\n' acoplada de manera síncrona con el operador static_cast de punto flotante.

☐ C.

El manipulador setw() de la librería en C++ y el método System.out.printf() con especificadores de formato en Java.

☐ D.

Únicamente la impresión recursiva de los caracteres residuales ('basura') almacenados en el Stack local.

Respuesta: C